

--- Ausgabe aktivieren

```
SET SERVEROUTPUT ON;
```

--- ROW LEVEL TRIGGER, declare würde for begin kommen

```
create or replace trigger parts_aft_ins_upd_row
after insert or update
on parts
for each row
begin
    if inserting then
        dbms_output.put_line('Insert on table parts detected.');
```

--- ROW LEVEL TRIGGER, auf alte/neue Werte zugreifen

```
create or replace trigger parts_aft_upd_color
after update of PartColor
on parts
for each row
begin
    dbms_output.put_line('Attention, part color changed from ' ||
        trim(:old.PartColor) || ' to ' ||
        trim(:new.PartColor) || '!');
```

--- ROW LEVEL TRIGGER, bedingte Ausführung

```
create or replace trigger parts_aft_upd_when
after update of PartColor
on parts
for each row when (new.PartColor = 'pink')
begin
    dbms_output.put_line('I am a trigger and I hate PINK!');
```

**--- STATEMENT TRIGGER: Speichere welcher User welche Änderung an der
--- Tabelle emp gemacht hat**

```
CREATE OR REPLACE TRIGGER AD_EMP
AFTER DELETE ON emp
DECLARE
    v_rowcnt number := 0;
    usr varchar2(50);
BEGIN
    SELECT count(*) INTO v_rowcnt FROM emp;
    SELECT user INTO usr from dual;
    INSERT INTO emp_count VALUES (usr,sysdate, v_rowcnt);
END;
```

--- CURSOR

```
declare
    pi constant number(9, 7) default 3.1415927;
    area number(14, 2);
    cursor radius_cursor is select * from radius_values;
begin
    for radius_row in radius_cursor loop
        area := pi * power(radius_row.radius, 2);
        dbms_output.put_line('A=' || AREA);
    end loop;
end;
```

--- EXCEPTION HANDLING

```
DECLARE
    pe_ratio NUMBER(3,1);
    own_failure exception;
BEGIN
    SELECT price / earnings INTO pe_ratio FROM stocks
        WHERE symbol = 'XYZ'; -- might cause division-by-zero error
    INSERT INTO stats (symbol, ratio) VALUES ('XYZ', pe_ratio);
    ...
    raise own_failure;
EXCEPTION -- exception handlers begin
    WHEN ZERO_DIVIDE THEN -- handles 'division by zero' error
        INSERT INTO stats (symbol, ratio) VALUES ('XYZ', NULL);
        COMMIT;
    ...
    WHEN OTHERS THEN -- handles all other errors
        ROLLBACK;
END;
```

--- Bedingte Anweisungen

```
if sales > 50000 then
    bonus := 1500;
elsif sales > 35000 then
    bonus := 500;
else
    bonus := 100;
end if;
```

--- UDF

```
create or replace function add_2_numbers(a number := 0, b number := 0)
return number
is
begin
    return a + b;
end;
```

```
select add_2_numbers(3, 11) from dual;
```

--- STP

```
create or replace procedure hello(name varchar)
as
begin
    dbms_output.put_line('Hello ' || name || '!');
end;
```

```
exec hello('Uriah');
```

-- Table-valued functions with collections

```
CREATE OR REPLACE TYPE t_tvf_row AS OBJECT (id NUMBER, description
VARCHAR2(50));
CREATE OR REPLACE TYPE t_tvf_table IS TABLE OF t_tvf_row;
```

```
CREATE OR REPLACE FUNCTION tvf_table_func (p_rows IN NUMBER)
RETURN t_tvf_table
AS
```

```
    l_tab t_tvf_table := t_tvf_table();
BEGIN
    FOR i IN 1 .. p_rows LOOP
        l_tab.extend;
        l_tab(l_tab.last) := t_tvf_row(i, 'Description for ' || i);
    END LOOP;
```

```
    RETURN l_tab;
END;
```

```
select * from tvf_table_func(10);
```